

Message Intelligence System - Full Project Report

Message Intelligence System

Spam vs. Legitimate Message Classification using Probability Theory & Machine Learning

Role: Data Scientist, Communication Security Company **Dataset:** `Message_Intelligence_Dataset.csv` — 5,199 messages **Target:** `spam_label` → 0 = Legitimate, 1 = Spam

Table of Contents

1. Objective & Problem Statement
 2. Part A — Probability & Conceptual Foundation (Theory)
 3. Part B — Dataset Understanding & Preparation
 4. Part C — Baseline Model: K-Nearest Neighbors
 5. Part D — Support Vector Machine Classifier
 6. Part E — Naive Bayes Classifier & Probability
 7. Part F — Model Comparison & Evaluation
 8. Part G — Final Analysis & Reporting
 9. Streamlit Application (Interactive Dashboard)
 10. Project Files & How to Run
 11. Appendix — Streamlit App Source Code (`app.py`)
-

1. Objective & Problem Statement

The objective of this project is to design a classification system that identifies whether incoming digital messages are **Spam** or **Legitimate**. The project combines probability theory with distance-based, margin-based, and probabilistic classifiers, and analyzes how different assumptions impact model performance.

As a Data Scientist for a communication security company, the task is to build a **Message Intelligence System** that automatically classifies user messages as:

- 0 → Legitimate Message
- 1 → Spam Message

The dataset contains message-related features extracted from text (numerical summaries) and user behavior signals. Some features are highly correlated, while others follow probabilistic patterns. The task is to build and compare multiple classification models and explain their performance using probability concepts.

2. Part A — Probability & Conceptual Foundation (Theory)

2.1 What is Conditional Probability?

Conditional probability is the probability of an event **A** occurring *given that* another event **B** has already occurred. It is written **P(A | B)** and defined as:

$$P(A | B) = P(A \cap B) / P(B), \text{ where } P(B) > 0$$

Example in this project: “What is the probability a message is spam, given that it contains a URL?” narrows the sample space from all messages to only those containing a URL.

2.2 Bayes’ Theorem and its Importance in Classification Problems

Bayes’ Theorem reverses the direction of a conditional probability — it lets us compute $P(\mathbf{A} \mid \mathbf{B})$ from the (often easier to estimate) $P(\mathbf{B} \mid \mathbf{A})$:

$$P(\mathbf{A} \mid \mathbf{B}) = [P(\mathbf{B} \mid \mathbf{A}) \times P(\mathbf{A})] / P(\mathbf{B})$$

For a classification problem with class $\mathbf{C}$ and observed features $\mathbf{X}$:

$$P(\mathbf{C} \mid \mathbf{X}) = [P(\mathbf{X} \mid \mathbf{C}) \times P(\mathbf{C})] / P(\mathbf{X})$$

Why it matters: It is usually easy to estimate $P(\mathbf{X} \mid \mathbf{C})$ (how features behave within each known class, from training data) and $P(\mathbf{C})$ (the class prior), but hard to estimate $P(\mathbf{C} \mid \mathbf{X})$ directly. Bayes’ Theorem converts these easy-to-learn quantities into the posterior probability we actually want: “given this message’s features, how likely is it spam?” This is the core mechanism behind the Naive Bayes classifier and underlies probabilistic reasoning in classification generally.

2.3 Assumptions Made by the Naive Bayes Classifier

- Conditional independence (“naive” assumption):** All features are assumed independent of one another *given* the class, i.e. $P(X_1, X_2, \dots, X_n \mid C) = P(X_1 \mid C) \times P(X_2 \mid C) \times \dots \times P(X_n \mid C)$. In reality, features such as `num_urls` and `spam_keyword_score` are often correlated, so this assumption is an approximation.
- Feature distribution assumption:** Gaussian Naive Bayes assumes each continuous feature is normally distributed within each class.
- Equal, independent contribution:** Each feature is assumed to contribute independently to the outcome — no interaction effects are modeled.
- Representative training data:** The training sample is assumed to be representative of the true class priors $P(\mathbf{C})$.

2.4 Working Principle of KNN and SVM

K-Nearest Neighbors (KNN): KNN is an instance-based, *distance-based* classifier with no explicit training phase (“lazy learner”). To classify a new point: (1) compute the distance (commonly Euclidean) from the new point to every point in the training set, (2) select the **K** closest neighbors, and (3) assign the **majority class** among them. It assumes that points close together in feature space tend to share the same label.

Support Vector Machine (SVM): SVM is a *margin-based* classifier. It searches for the hyperplane that separates the two classes while **maximizing the margin** — the distance between the hyperplane and the nearest points of each class, called the **support vectors**. When classes aren’t linearly separable, a **kernel trick** (e.g., RBF or polynomial) implicitly maps the data into a higher-dimensional space where a linear separator can be found.

2.5 Comparing Distance-Based, Probabilistic, and Margin-Based Classifiers

Aspect	Distance-based (KNN)	Probabilistic (Naive Bayes)	Margin-based (SVM)
Core idea	Majority vote of nearest neighbors	Posterior class probability via Bayes’ Theorem	Maximizes separating margin between classes
Training cost	None (lazy learner)	Fast — estimates means/variances/priors	Can be expensive (quadratic optimization)
Prediction cost	Expensive (compares to all training points)	Very fast	Fast (depends only on support vectors)
Key assumption	Similar points share similar labels	Feature independence given class	Classes are separable (linearly or via kernel)
Sensitive to scaling	Yes, strongly	Less sensitive	Yes, strongly

Interpretability	Low (no explicit model)	High (explicit probabilities)	Medium
High-dimensional data	Struggles (curse of dimensionality)	Handles well	Handles well, especially with kernels

2.6 Evaluation Metrics Used in This Project

- **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$ — overall fraction of correct predictions.
- **Precision** = $TP / (TP + FP)$ — of all messages predicted spam, how many actually were spam. High precision minimizes false alarms (legitimate messages wrongly blocked).
- **Recall** = $TP / (TP + FN)$ — of all actual spam messages, how many were correctly caught. High recall minimizes missed spam (security risk).
- **F1 Score** = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ — harmonic mean balancing precision and recall.

(*TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative; “positive” = spam*)

3. Part B — Dataset Understanding & Preparation

The dataset (`Message_Intelligence_Dataset.csv`) contains **5,199 messages** with the following columns:

Column	Description
<code>message_id</code>	Unique identifier
<code>message_text</code>	Raw message text
<code>message_length</code> , <code>word_count</code>	Size of the message
<code>num_urls</code> , <code>num_digits</code> , <code>num_special_chars</code>	Content/structural signals
<code>spam_keyword_score</code> , <code>legit_keyword_score</code>	Keyword-based signals
<code>sender_activity_score</code> , <code>sender_account_age_days</code> , <code>messages_sent_last_24h</code>	User behavior signals
<code>timestamp</code> , <code>hour_of_day</code> , <code>day_of_week</code>	Temporal features
<code>spam_label</code>	Target — 0 = Legitimate, 1 = Spam

Tasks completed:

- **Input features / target identification:** 12 numeric features identified as inputs; `spam_label` as target.
- **Missing values:** `sender_activity_score` (106), `sender_account_age_days` (113), and `messages_sent_last_24h` (162) contained missing values — handled with **median imputation**.
- **Scaling:** Features standardized with `StandardScaler` for the distance-based (KNN) and margin-based (SVM) models. Naive Bayes used unscaled features (Gaussian NB is not scale-sensitive).
- **Train/test split:** 80/20 stratified split (`random_state=42`) to preserve the original class balance.
- **Class balance:** ~81.3% Legitimate vs. ~18.7% Spam — a moderately imbalanced dataset, which is why Precision/Recall/F1 (not just Accuracy) were prioritized in evaluation.
- **Correlation analysis:** A correlation heatmap showed several features (e.g., `num_urls`, `spam_keyword_score`, `num_special_chars`) moving together for spam messages — informing the discussion of the Naive Bayes independence assumption in Part G.

4. Part C — Baseline Model: K-Nearest Neighbors (Distance-Based)

KNN was implemented and tested across a sweep of **K** values (1 to 21) using Euclidean distance:

K	Accuracy	Precision	Recall	F1
1	1.000	1.0	1.0000	1.0000
3	1.000	1.0	1.0000	1.0000
5	1.000	1.0	1.0000	1.0000

7-21	0.999	1.0	0.9949	0.9974
------	-------	-----	--------	--------

Best K (by F1 score): K = 1, 3, or 5 (all tied at perfect F1 = 1.0000).

Distance metric comparison (at the best K):

Distance Metric	Accuracy	Precision	Recall	F1
Euclidean	1.000	1.0	1.0000	1.0000
Manhattan	1.000	1.0	1.0000	1.0000
Chebyshev	0.999	1.0	0.9949	0.9974

Interpretation: Euclidean treats all feature differences as squared (sensitive to large deviations in a single feature), Manhattan sums absolute differences (more robust to single-feature outliers), and Chebyshev only considers the single maximum-difference feature. Because the scaled features include several skewed counts (URLs, digits), the choice of metric shifts which neighbors are considered “closest,” slightly changing borderline predictions — visible in Chebyshev’s marginally lower recall.

Misclassification analysis: A small number of test messages were misclassified by KNN at higher K values — these tended to be borderline messages with mixed signals (e.g., legitimate messages containing a URL, or spam messages with low keyword scores).

5. Part D — Support Vector Machine Classifier (Margin-Based)

Both **Linear** and **RBF kernel** SVMs were implemented:

Model	Accuracy	Precision	Recall	F1	Support Vectors
Linear SVM	1.0000	1.0000	1.0000	1.0000	10
RBF SVM	1.0000	1.0000	1.0000	1.0000	138

Margin & support vector analysis: The Linear SVM required only **10 support vectors** ($\approx 0.2\%$ of training data) to achieve perfect separation — indicating the classes are very well-separated linearly in this feature space, with a wide margin. The RBF kernel used **138 support vectors**, reflecting its more flexible, non-linear boundary that hugs the data more closely (still achieving the same perfect test performance here, but at higher model complexity).

Comparison with KNN: All three models (KNN, Linear SVM, RBF SVM) achieved effectively perfect performance (Accuracy = F1 = 1.0000) on this dataset’s test split — suggesting the engineered features make spam vs. legitimate messages highly separable, regardless of which decision boundary shape is used.

6. Part E — Naive Bayes Classifier & Probability (Probabilistic)

Gaussian Naive Bayes was implemented on unscaled features:

Metric	Value
Accuracy	0.9990
Precision	0.9949
Recall	1.0000
F1 Score	0.9974

Learned class priors: $P(\text{Legitimate}) = 0.8130$, $P(\text{Spam}) = 0.1870$ — matching the dataset’s natural class balance.

Manual Bayes’ Theorem computation: For sample test messages, conditional probabilities were computed by hand using the Gaussian probability density function for each feature, combined as:

$$P(C | X) \propto P(C) \times \prod_i P(x_i | C)$$

The resulting posterior probabilities (e.g., $P(\text{Legitimate} | X)$ vs. $P(\text{Spam} | X)$) were normalized and compared directly against scikit-learn's `predict_proba()` output — **the manual calculations matched the model's output within floating-point precision**, confirming correct understanding and implementation of Bayes' Theorem in this context.

7. Part F — Model Comparison & Evaluation

Final comparison table:

Model	Type	Accuracy	Precision	Recall	F1 Score
KNN	Distance-based	1.000	1.0000	1.0000	1.0000
Linear SVM	Margin-based	1.000	1.0000	1.0000	1.0000
RBF SVM	Margin-based	1.000	1.0000	1.0000	1.0000
Naive Bayes	Probabilistic	0.999	0.9949	1.0000	0.9974

Best model by use case:

- **Highest Precision:** KNN / SVM (tied at 1.0000) — best when the cost of a **false positive** (blocking a real message) is high.
- **Highest Recall:** All models effectively tied at 1.0000 — minimal risk of letting spam through undetected.
- **Best overall balance (F1):** KNN / SVM (tied at 1.0000), with Naive Bayes very close behind (0.9974).

8. Part G — Final Analysis & Reporting

Strengths and Weaknesses of Each Classifier

Model	Strengths	Weaknesses
KNN	Simple, no training phase, intuitive, performed perfectly here	Slow at prediction time on large datasets, sensitive to feature scaling and choice of K/metric, struggles in high dimensions
Linear SVM	Strong margin-based separation, very few support vectors (excellent generalization), fast prediction	Cannot capture non-linear boundaries, sensitive to scaling
RBF SVM	Captures non-linear boundaries, flexible	More hyperparameters to tune, harder to interpret, more support vectors → slower prediction, overfitting risk if mistuned
Naive Bayes	Extremely fast, naturally probabilistic, no scaling needed, highly interpretable	Independence assumption rarely holds exactly; slightly lower precision here due to that approximation

Impact of Probability Assumptions in Naive Bayes

The correlation heatmap from Part B showed noticeable correlation between features such as `num_urls`, `spam_keyword_score`, and `num_special_chars` — these tend to move together in spam messages. Violating Naive Bayes's independence assumption means it can “double count” evidence from correlated features, biasing the posterior (often making it overconfident). This shows up subtly in the results: Naive Bayes' precision (0.9949) is slightly lower than KNN/SVM's perfect precision, meaning it produced a few more false positives — a plausible consequence of the independence assumption being only approximately true.

Trade-offs Between Interpretability and Performance

- **Naive Bayes** is the *most interpretable* — every prediction can be explained as a comparison of explicit, human-readable probabilities — but it sacrifices a small amount of precision due to violated

independence assumptions.

- **KNN** is conceptually simple but offers no learned model to inspect — explanations come only from examining which neighbors drove a decision.
- **SVM (especially RBF)** delivers strong raw performance but is the *least interpretable*, since its boundary lives in an implicit, kernel-transformed space.

In short: on this dataset, near-perfect performance was achievable across all three model families, but **Naive Bayes remains the best choice when explainability is the priority**, while **KNN/SVM edge out marginally on raw precision/recall**.

Business Recommendation for Real-World Deployment

1. **Primary classifier — Linear SVM:** strong, perfectly-separating, and uses very few support vectors, which is computationally cheap to deploy.
2. **Secondary, explainable layer — Naive Bayes:** run alongside the primary model to generate human-readable probability scores for support agents and manual-review cases.
3. **Threshold tuning by business cost:** tune the classification threshold toward higher precision if user trust/inbox experience is the priority, or toward higher recall if security/phishing prevention is the priority.
4. **Monitoring & retraining:** spam tactics evolve quickly, so the chosen model should be retrained on a rolling window of recent messages.
5. **Avoid KNN at scale:** despite its strong accuracy here, KNN's prediction-time cost (comparing against the full training set per query) makes it a poor fit for a high-throughput, real-time message-filtering system in production.

9. Streamlit Application (Interactive Dashboard)

To make the analysis interactive and accessible to non-technical stakeholders, the project includes a **Streamlit web app** (`app.py`) with five tabs:

1. 📄 **Overview** — dataset snapshot, class balance metric cards, feature list.
2. 🔍 **EDA** — interactive Plotly charts: class distribution (bar + pie), correlation heatmap, per-feature distribution histograms (by class), and a 2D PCA scatter plot.
3. 🛠️ **Models** — adjustable hyperparameters (K & distance metric for KNN; kernel, C, gamma for SVM) via the sidebar, live metrics, interactive confusion matrices, classification reports, and an ROC curve comparison across all three models.
4. 📊 **Comparison** — a grouped bar chart and table comparing Accuracy/Precision/Recall/F1 across models, with automatic “best model for X” callouts.
5. ✉️ **Try a Message** — load a random test message or manually enter feature values, then get a live prediction (plus Naive Bayes posterior probabilities) from all three trained models simultaneously.

The app uses `@st.cache_data` / `@st.cache_resource` to avoid retraining models on every interaction, and all visualizations use **Plotly** for hover/zoom interactivity.

10. Project Files & How to Run

Repository contents:

```
.
├── Message_Intelligence_System.ipynb    # Full notebook (Parts B-G)
├── Message_Intelligence_Dataset.csv     # Source dataset
├── Theory_Concepts.pdf                  # Part A theory (standalone)
├── app.py                              # Streamlit interactive dashboard
├── requirements.txt                    # Python dependencies
└── README.md                           # Project documentation
```

To run the Jupyter Notebook:

```
pip install pandas numpy matplotlib seaborn scikit-learn scipy
jupyter notebook Message_Intelligence_System.ipynb
```

To run the Streamlit App:

```
pip install -r requirements.txt
streamlit run app.py
```

requirements.txt contents:

```
streamlit==1.39.0
pandas==2.2.3
numpy==1.26.4
scikit-learn==1.5.2
plotly==5.24.1
scipy==1.13.1
```

11. Appendix — Streamlit App Source Code (`app.py`)

```
"""
Message Intelligence System — Streamlit App
Spam vs Legitimate message classification using KNN, SVM, and Naive Bayes.

Run with:
    pip install streamlit pandas numpy scikit-learn matplotlib seaborn
    streamlit run app.py
"""

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import streamlit as st

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, auc
)

# -----
# Page config
# -----
st.set_page_config(page_title="Message Intelligence System", layout="wide", page_icon="📧")

FEATURE_COLS = [
    'message_length', 'word_count', 'num_urls', 'num_digits',
    'num_special_chars', 'spam_keyword_score', 'legit_keyword_score',
    'sender_activity_score', 'sender_account_age_days',
    'messages_sent_last_24h', 'hour_of_day', 'day_of_week'
]

TARGET_COL = 'spam_label'

# -----
# Cached data + model helpers
# -----
@st.cache_data
def load_data(path_or_buffer):
    df = pd.read_csv(path_or_buffer)
    return df

@st.cache_data
def preprocess(df, test_size, random_state):
    X = df[FEATURE_COLS].copy()
    y = df[TARGET_COL].copy()

    # Median-impute any missing numeric values
    for c in X.columns[X.isna().any()]:
        X[c] = X[c].fillna(X[c].median())

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    scaler = StandardScaler()
```

```

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

return X, y, X_train, X_test, y_train, y_test, X_train_scaled, X_test_scaled, scaler

@st.cache_resource
def train_models(X_train_scaled, X_train, y_train, knn_k, knn_metric, svm_kernel, svm_C, svm_gamma):
    knn = KNeighborsClassifier(n_neighbors=knn_k, metric=knn_metric)
    knn.fit(X_train_scaled, y_train)

    svm = SVC(kernel=svm_kernel, C=svm_C, gamma=svm_gamma, probability=True, random_state=42)
    svm.fit(X_train_scaled, y_train)

    nb = GaussianNB()
    nb.fit(X_train, y_train) # Naive Bayes uses unscaled features

    return knn, svm, nb

def get_metrics(y_true, y_pred):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1 Score": f1_score(y_true, y_pred, zero_division=0),
    }

def plot_confusion_plotly(y_true, y_pred, title, colorscale="Blues"):
    cm = confusion_matrix(y_true, y_pred)
    labels = ['Legit', 'Spam']
    fig = px.imshow(
        cm, text_auto=True, color_continuous_scale=colorscale,
        x=labels, y=labels, labels=dict(x="Predicted", y="Actual", color="Count")
    )
    fig.update_layout(title=title, height=350, margin=dict(t=50, b=10, l=10, r=10))
    return fig

def plot_roc_plotly(models_preds_proba, y_true, title="ROC Curve Comparison"):
    fig = go.Figure()
    for name, proba in models_preds_proba.items():
        fpr, tpr, _ = roc_curve(y_true, proba)
        roc_auc = auc(fpr, tpr)
        fig.add_trace(go.Scatter(x=fpr, y=tpr, mode='lines', name=f"{name} (AUC={roc_auc:.3f})"))
    fig.add_trace(go.Scatter(x=[0, 1], y=[0, 1], mode='lines', name="Random",
        line=dict(dash='dash', color='gray'))))
    fig.update_layout(title=title, xaxis_title="False Positive Rate",
        yaxis_title="True Positive Rate", height=450)
    return fig

# -----
# Sidebar - data source & settings
# -----
st.sidebar.title("⚙ Settings")

uploaded = st.sidebar.file_uploader("Upload dataset CSV", type=["csv"])
default_path = "Message_Intelligence_Dataset.csv"

if uploaded is not None:
    df = load_data(uploaded)
elif __import__("os").path.exists(default_path):
    df = load_data(default_path)
else:
    st.sidebar.warning("Upload `Message_Intelligence_Dataset.csv` to get started.")
    st.stop()

st.sidebar.markdown("---")
st.sidebar.subheader("Train/Test Split")
test_size = st.sidebar.slider("Test size", 0.1, 0.4, 0.2, 0.05)
random_state = st.sidebar.number_input("Random state", value=42, step=1)

st.sidebar.subheader("KNN Hyperparameters")
knn_k = st.sidebar.slider("K (neighbors)", 1, 25, 7, 2)
knn_metric = st.sidebar.selectbox("Distance metric", ["euclidean", "manhattan", "chebyshev"])

st.sidebar.subheader("SVM Hyperparameters")
svm_kernel = st.sidebar.selectbox("Kernel", ["rbf", "linear", "poly"])
svm_C = st.sidebar.slider("C (regularization)", 0.01, 10.0, 1.0, 0.01)
svm_gamma = st.sidebar.selectbox("Gamma", ["scale", "auto"])

X, y, X_train, X_test, y_train, y_test, X_train_scaled, X_test_scaled, scaler = preprocess(
    df, test_size, random_state
)
knn, svm, nb = train_models(
    X_train_scaled, X_train, y_train, knn_k, knn_metric, svm_kernel, svm_C, svm_gamma
)

```



```

knn_pred = knn.predict(X_test_scaled)
svm_pred = svm.predict(X_test_scaled)
nb_pred = nb.predict(X_test)

# -----
# Header
# -----
st.title("📧 Message Intelligence System")
st.caption("Spam vs Legitimate message classification — KNN (distance-based) · SVM (margin-based) · Naive Bayes (probabilistic)")

tab_overview, tab_eda, tab_models, tab_compare, tab_predict = st.tabs(
    ["📄 Overview", "🔍 EDA", "🧠 Models", "📊 Comparison", "📧 Try a Message"]
)

# -----
# TAB: Overview
# -----
with tab_overview:
    st.subheader("Dataset Snapshot")
    c1, c2, c3 = st.columns(3)
    c1.metric("Total messages", len(df))
    c2.metric("Spam (%)", f"{(df[TARGET_COL].mean() * 100):.1f}%")
    c3.metric("Features used", len(FEATURE_COLS))

    st.dataframe(df.head(20), use_container_width=True)

    st.markdown("""
    **Target variable:** `spam_label` → `0` = Legitimate, `1` = Spam

    **Features:** message length, word count, URL/digit/special-character counts,
    spam/legit keyword scores, sender activity score, sender account age,
    messages sent in last 24h, hour of day, day of week.
    """)

# -----
# TAB: EDA
# -----
with tab_eda:
    st.subheader("Class Distribution")
    col1, col2 = st.columns(2)

    counts = df[TARGET_COL].value_counts().rename({0: "Legitimate", 1: "Spam"})

    with col1:
        fig_bar = px.bar(
            x=counts.index, y=counts.values, color=counts.index,
            color_discrete_map={"Legitimate": "#4C72B0", "Spam": "#DD8452"},
            labels={"x": "Class", "y": "Count"}, title="Class Counts", text=counts.values
        )
        fig_bar.update_layout(showlegend=False, height=380)
        st.plotly_chart(fig_bar, use_container_width=True)

    with col2:
        fig_pie = px.pie(
            names=counts.index, values=counts.values,
            color=counts.index,
            color_discrete_map={"Legitimate": "#4C72B0", "Spam": "#DD8452"},
            title="Class Proportion", hole=0.35
        )
        fig_pie.update_layout(height=380)
        st.plotly_chart(fig_pie, use_container_width=True)

    st.subheader("Feature Correlation Heatmap")
    corr = df[FEATURE_COLS + [TARGET_COL]].corr().round(2)
    fig_corr = px.imshow(
        corr, text_auto=True, color_continuous_scale="RdBu_r", zmin=-1, zmax=1,
        aspect="auto"
    )
    fig_corr.update_layout(height=600, margin=dict(t=30, b=10, l=10, r=10))
    st.plotly_chart(fig_corr, use_container_width=True)

    st.subheader("Feature Distributions by Class")
    feat_choice = st.selectbox("Choose a feature", FEATURE_COLS)
    plot_df = df.copy()
    plot_df['Label'] = plot_df[TARGET_COL].map({0: "Legitimate", 1: "Spam"})
    fig_hist = px.histogram(
        plot_df, x=feat_choice, color="Label", barmode="overlay", marginal="box",
        color_discrete_map={"Legitimate": "#4C72B0", "Spam": "#DD8452"},
        opacity=0.7, nbins=40
    )
    fig_hist.update_layout(height=450)
    st.plotly_chart(fig_hist, use_container_width=True)

    st.subheader("2D PCA Projection (colored by class)")
    pca_viz = PCA(n_components=2, random_state=42)
    proj =
    pca_viz.fit_transform(StandardScaler().fit_transform(df[FEATURE_COLS].fillna(df[FEATURE_COLS].median())))
    proj_df = pd.DataFrame(proj, columns=["PC1", "PC2"])
    proj_df['Label'] = df[TARGET_COL].map({0: "Legitimate", 1: "Spam"}).values

```

```

fig_pca = px.scatter(
    proj_df, x="PC1", y="PC2", color="Label",
    color_discrete_map={"Legitimate": "#4C72B0", "Spam": "#DD8452"},
    opacity=0.6, title=f"PCA (explained variance: {pca_viz.explained_variance_ratio_.sum()*100:.1f}%)"
)
fig_pca.update_layout(height=450)
st.plotly_chart(fig_pca, use_container_width=True)

# -----
# TAB: Models
# -----
with tab_models:
    st.subheader("Individual Model Performance")

    m1, m2, m3 = st.tabs(["KNN", "SVM", "Naive Bayes"])

    with m1:
        st.markdown(f"K = {knn_k}, metric = {knn_metric}")
        metrics = get_metrics(y_test, knn_pred)
        cols = st.columns(4)
        for col, (k, v) in zip(cols, metrics.items()):
            col.metric(k, f"{v:.3f}")
        st.plotly_chart(plot_confusion_plotly(y_test, knn_pred, "KNN Confusion Matrix", "Blues"),
            use_container_width=True)
        with st.expander("Classification report"):
            st.text(classification_report(y_test, knn_pred, target_names=['Legitimate', 'Spam']))

    with m2:
        st.markdown(f"Kernel = {svm_kernel}, C = {svm_C}, gamma = {svm_gamma}")
        metrics = get_metrics(y_test, svm_pred)
        cols = st.columns(4)
        for col, (k, v) in zip(cols, metrics.items()):
            col.metric(k, f"{v:.3f}")
        st.write(f"Support vectors: **{svm.support_vectors_.shape[0]** "
            f"{svm.support_vectors_.shape[0] / len(X_train_scaled) * 100:.1f}% of training data)")
        st.plotly_chart(plot_confusion_plotly(y_test, svm_pred, "SVM Confusion Matrix", "Oranges"),
            use_container_width=True)
        with st.expander("Classification report"):
            st.text(classification_report(y_test, svm_pred, target_names=['Legitimate', 'Spam']))

    with m3:
        st.markdown("Gaussian Naive Bayes (unscaled features)")
        metrics = get_metrics(y_test, nb_pred)
        cols = st.columns(4)
        for col, (k, v) in zip(cols, metrics.items()):
            col.metric(k, f"{v:.3f}")
        st.write(f"Class priors → P(Legitimate) = {nb.class_prior_[0]:.3f}, "
            f"P(Spam) = {nb.class_prior_[1]:.3f}")
        st.plotly_chart(plot_confusion_plotly(y_test, nb_pred, "Naive Bayes Confusion Matrix", "Greens"),
            use_container_width=True)
        with st.expander("Classification report"):
            st.text(classification_report(y_test, nb_pred, target_names=['Legitimate', 'Spam']))

    st.subheader("ROC Curve Comparison")
    proba_dict = {
        "KNN": knn.predict_proba(X_test_scaled)[ :, 1],
        "SVM": svm.predict_proba(X_test_scaled)[ :, 1],
        "Naive Bayes": nb.predict_proba(X_test)[ :, 1],
    }
    st.plotly_chart(plot_roc_plotly(proba_dict, y_test), use_container_width=True)

# -----
# TAB: Comparison
# -----
with tab_compare:
    st.subheader("Model Comparison")

    results = pd.DataFrame({
        'Model': ['KNN', 'SVM', 'Naive Bayes'],
        'Type': ['Distance-based', 'Margin-based', 'Probabilistic'],
        **{k: [get_metrics(y_test, p)[k] for p in [knn_pred, svm_pred, nb_pred]]
            for k in ['Accuracy', 'Precision', 'Recall', 'F1 Score']}
    }).round(4)

    st.dataframe(results, use_container_width=True)

    melted = results.melt(id_vars=['Model', 'Type'], value_vars=['Accuracy', 'Precision', 'Recall', 'F1 Score'],
        var_name='Metric', value_name='Score')

    fig_cmp = px.bar(
        melted, x='Model', y='Score', color='Metric', barmode='group',
        text=melted['Score'].round(3), title="Accuracy / Precision / Recall / F1 by Model"
    )
    fig_cmp.update_layout(yaxis_range=[0, 1.05], height=480)
    st.plotly_chart(fig_cmp, use_container_width=True)

    best_precision = results.loc[results['Precision'].idxmax()]
    best_recall = results.loc[results['Recall'].idxmax()]
    best_f1 = results.loc[results['F1 Score'].idxmax()]

```

```

c1, c2, c3 = st.columns(3)
c1.info(f"""Best Precision:** {best_precision['Model']} ({best_precision['Precision']:.3f})\n\n"
        "Use when false positives (blocking real messages) are costly.")
c2.info(f"""Best Recall:** {best_recall['Model']} ({best_recall['Recall']:.3f})\n\n"
        "Use when missing actual spam (security risk) is costly.")
c3.success(f"""Best F1 (balanced):** {best_f1['Model']} ({best_f1['F1 Score']:.3f})")

# -----
# TAB: Try a Message (live prediction)
# -----
with tab_predict:
    st.subheader("Classify a Custom Message")
    st.caption("Enter feature values manually, or pick a random message from the test set to auto-fill.")

    if st.button("🔄 Load random test message"):
        sample = X_test.sample(1, random_state=np.random.randint(0, 10000))
        st.session_state["sample_idx"] = sample.index[0]

    sample_idx = st.session_state.get("sample_idx", X_test.index[0])
    sample_row = X_test.loc[sample_idx]
    actual_label = y_test.loc[sample_idx]

    if 'message_text' in df.columns:
        st.text_area("Sample message text", df.loc[sample_idx, 'message_text'], height=70, disabled=True)
        st.write(f"Actual label: **{'Spam' if actual_label == 1 else 'Legitimate'}**")

    cols = st.columns(3)
    input_vals = {}
    for i, feat in enumerate(FEATURE_COLS):
        with cols[i % 3]:
            input_vals[feat] = st.number_input(
                feat, value=float(sample_row[feat]), key=f"inp_{feat}"
            )

    if st.button("🔍 Classify Message", type="primary"):
        x_input = pd.DataFrame([input_vals])[FEATURE_COLS]
        x_input_scaled = scaler.transform(x_input)

        knn_p = knn.predict(x_input_scaled)[0]
        svm_p = svm.predict(x_input_scaled)[0]
        nb_p = nb.predict(x_input)[0]
        nb_proba = nb.predict_proba(x_input)[0]

        r1, r2, r3 = st.columns(3)
        for col, name, pred in zip([r1, r2, r3], ["KNN", "SVM", "Naive Bayes"], [knn_p, svm_p, nb_p]):
            label = "🚫 Spam" if pred == 1 else "✅ Legitimate"
            col.metric(name, label)

        st.write(f"""Naive Bayes posterior probabilities:** "
                f"P(Legitimate) = {nb_proba[0]:.4f}, P(Spam) = {nb_proba[1]:.4f}")

    st.markdown("---")
    st.caption("Message Intelligence System · Built with Streamlit, scikit-learn, pandas & seaborn")

```

Message Intelligence System — “Quality is our motto.”